

SMART FARMER ASSISTANCE

System Architecture, Technical Logic & Technology Specification Guide

A comprehensive technical reference detailing the full-stack architecture, end-to-end operational algorithms, modular domain decomposition, API integrations, and database schemas of the Smart Farmer Mandi Procurement Platform.

Version: v2.4 Production Release	Published: 16 September 2026	Platform: TypeScript Monorepo (Web, Coverage: Core Logic, APMC Gate Engine, Tulai Parchi, Telegram PDF, Socket Sync, AI & APIs REST, WS)
---	-------------------------------------	---

DOCUMENT TABLE OF CONTENTS

1. Executive Summary & Architecture Overview	— Monorepo layout, high-level tiers, design philosophy
2. End-to-End Operational Logic & User Journeys	— Gate entry, token allocation, moisture assay, Tulai Parchi & DBT
3. Codebase Structure & Functional Module Mapping	— Which specific files and controllers execute which functions
4. Complete Technology Stack & Purpose of Each Tool	— Frontend, backend, ORM, validation, and real-time drivers
5. External APIs, Telegram Bot & Integration Services	— IMD weather, Telegram PDF dispatch, dynamic translations, Agmarknet
6. Data Integrity, Concurrency & Security Standards	— ACID transactions, silo capacity locking, JWT auth, and verification

1. EXECUTIVE SUMMARY & SYSTEM ARCHITECTURE

The Smart Farmer Assistance Platform is a specialized digital public infrastructure solution engineered for agricultural mandis (APMCs), smallholder farmers, and procurement officers across India. Its core purpose is to eradicate predatory middleman practices, long physical tractor queues, arbitrary moisture-based price cuts, and opaque weighing by digitizing the agricultural supply chain from pre-harvest scheduling to direct bank disbursement.

High-Level Monorepo Structure

The project is architected as an npm-workspace monorepo with three strictly decoupled layers:

- **@smart-farmer/shared:** Houses cross-cutting TypeScript interfaces, Data Transfer Objects (DTOs), Zod validation schemas, and domain enum types (QueueStatus, CentreStatus, SchemeCategory). Both client and server import from this shared contract library, guaranteeing zero runtime type divergence.
- **smart-farmer-server:** A high-throughput Node.js & Express service running on TypeScript with Prisma ORM. It manages relational persistence, WebSocket lifecycle events, automated PDF generation, Telegram bot alerts, and APMC regulatory business rules.
- **smart-farmer-client:** A Single Page Application (SPA) built with React 18, Vite, and Tailwind CSS. It delivers domain-driven sub-portals for Farmers, Mandi Managers, and State Agricultural Administrators with zero visual regressions.

Architectural Decoupling Principle

All domain boundaries follow clean architecture: presentation components never make raw SQL calls, controllers remain thin delegators, business calculations (moisture refraction, wait times, silo headroom) reside in services, and stateful synchronization relies on typed WebSocket events.

2. END-TO-END OPERATIONAL LOGIC & WORKFLOWS

The platform automates the multi-stage agricultural procurement lifecycle. The entire flow runs on real-time event-driven algorithms outlined below:

Stage 1: Farmer Slot Reservation & Virtual Token Issuance

When a farmer prepares to bring produce (e.g. 50 Quintals of Wheat) to an APMC centre, they book an entry slot via the Farmer Portal:

- **Proximity & Capacity Routing:** The system queries available APMC yards filtered by state/district and sorts by remaining storage capacity and intake processing rate (Qtl/hour).
- **Sequential Token Generation:** Upon submission, the QueueService generates an idempotent token number formatted as T-YYYY-XXXX (e.g., T-2026-0042) and assigns status WAITING.
- **Dynamic Wait Time Estimation Algorithm:** The expected wait time in minutes is computed using real-time mandi telemetry: $\text{EstWaitMinutes} = (\text{FarmersAhead} * \text{AverageConsignmentWeight}) / (\text{MandiProcessingRate} / 60)$.

Stage 2: Mandi Gate Entry & Vehicle Arrival

When the farmer arrives at the physical APMC gate in their tractor trolley or commercial truck, the mandi gatekeeper registers arrival. The queue table moves the farmer into the physical waiting pool and updates all farmers ahead via live WebSocket broadcast.

Stage 3: Active Inspection Bay & Moisture Quality Assay

The Centre In-Charge calls the farmer to Weighing Bay 1. Here, statutory Fair Average Quality (FAQ) norms are applied:

- **Moisture Refraction Cutoff:** Under statutory APMC regulations, wheat moisture is benchmarked at 12.0%. Consignments under 12.0% pass with Grade A (100% MSP). Moisture between 12.1% and 14.0% incurs a value cut (refraction deduction). Consignments exceeding 14.0% are rejected.
- **Foreign Matter & Impurities:** Dust, chaff, and damaged grains are analyzed. The assay modal automatically calculates gross deductions before final weighment lock-in.

Stage 4: Tulai Parchi (Weighment Slip) & Disbursal

- **Net Payment Calculation:** $\text{Net Payable} = (\text{Actual Quintals} * \text{Statutory MSP Rate}) - \text{Moisture/Foreign Matter Deductions}$.
- **Cryptographic Receipt ID:** Generates an immutable receipt record: PAY-YYYYMMDD-TOKEN-RANDOM (e.g., PAY-20260916-0042-8712).
- **Vector PDF Generation:** The ReceiptService generates a tamper-evident digital Tulai Parchi with APMC government watermark, batch metadata, and payment breakdown.

Stage 5: Instant Telegram Notification Dispatch

Immediately after weighment confirmation, the TelegramService streams the generated PDF document and sends an instant SMS/Telegram push message to the farmer smartphone containing token number, net amount, and bank account mask.

3. CODEBASE STRUCTURE & FUNCTIONAL MODULE MAPPING

The table below maps the specific files and folders across the codebase to the exact functions they execute:

server/src/server.ts

Express app setup, HTTP server creation, Socket.IO binding, global middlewares, CORS config, and database connection handling.

server/src/controllers/queue.controller.ts

REST handlers for token booking, token calling, status advance, absent skipping, and weighment completion.

server/src/controllers/payment.controller.ts

Financial transaction queries, farmer settlement history, downloadable receipt streams, and DBT records.

server/src/controllers/scheme.controller.ts

Government welfare scheme publishing, statewide amendments, category queries, and deletion broadcasting.

server/src/services/queue.service.ts

Core APMC queue engine: slot reservation, sequential numbering, queue positioning, assay completion, and capacity decrement.

server/src/services/receipt.service.ts

Server-side PDFKit document builder for generating official bilingual APMC Mandi Weighment Slips (Tulai Parchi).

server/src/services/telegram.service.ts

Telegraf bot automation for dispatching transaction confirmations and sending PDF weighment slips as attachments.

server/src/services/ai.service.ts

Rule-based and LLM-assisted agronomy advisor, crop disease diagnosis, weather advisories, and mandi price predictions.

server/src/services/scheme.service.ts

Welfare policy CRUD, state filtering, and Socket.IO broadcast trigger on policy updates.

server/src/sockets/index.ts

WebSocket connection rooms, centre room subscriptions (centre:CENTRE_ID), and real-time broadcasting of queue and silo events.

server/src/utils/constants.ts

Central APMC regulatory standards: FAQ moisture limits (Wheat 12%, Paddy 17%), quality grade tiers, and vehicle norms.

client/src/components/farmer/ActiveTokenCard.tsx

Live token tracker card showing token number, queue position, farmers ahead, estimated wait time, and gate pass.

client/src/components/farmer/FarmerWeatherCard.tsx

Telemetry component displaying microclimate conditions, humidity, precipitation, and agricultural spraying advisories.

client/src/components/farmer/FarmerBankDetailsModal.tsx

Secure modal allowing farmers to view and edit Aadhaar-linked DBT direct deposit bank credentials.

client/src/components/manager/CapacityRateController.tsx

Interactive mandi intake controller with occupancy gauges, headroom calculator, and processing rate sliders.

client/src/components/manager/VehicleQueueTable.tsx

High-density manager table displaying vehicle arrivals in chronological order with one-click gate actions.

client/src/components/manager/ActiveInspectionBay.tsx

Weighing Bay 1 inspection panel managing called farmers, start weighing triggers, and quality assay initiation.

client/src/components/manager/FarmerIdentityModal.tsx

Farmer verification modal featuring profile photo check, direct phone call, and WhatsApp communication shortcuts.

client/src/components/admin/SchemeFormModal.tsx

Administrative policy editor for creating and updating welfare programs with live statewide push broadcast.

client/src/hooks/useDynamicTranslation.ts

High-performance React hook managing batch translation, in-memory caching, deduplication, and chunking.

client/src/utils/formatters.ts

Standardized formatters for Indian Rupee (INR), quintal weights, dates, and masked bank account numbers.

4. COMPLETE TECHNOLOGY STACK & PURPOSE OF EACH TOOL

Every technology in the stack was selected to solve specific real-world agrarian constraints: high concurrency during harvest peaks, intermittent rural connectivity, multi-lingual accessibility, and legal auditability.

TypeScript 5.x [End-to-End Type Safety]

Used across client, server, and shared libraries to guarantee strict typing. Eliminates undefined property crashes, ensures DTO contract compliance, and accelerates refactoring without regressions.

React 18 & Vite [Modern Frontend Framework & Bundler]

Provides component-based architecture for rich dashboards. Vite provides sub-second Hot Module Replacement (HMR) and optimized Rollup production builds with granular chunking.

Tailwind CSS [Utility-First Styling System]

Enables a clean, responsive interface matching modern human-designed standards. Eliminates CSS bloat, provides consistent color palettes (emerald, slate, amber), and supports mobile-first design for rural smartphones.

Node.js & Express.js [Asynchronous Backend Runtime & API Framework]

Leverages the non-blocking event-driven I/O model of Node.js to handle thousands of concurrent farmer requests, live WebSocket connections, and real-time intake telemetry updates with low memory overhead.

Prisma ORM [Next-Generation Database Client]

Provides auto-generated, type-safe database access for SQLite/PostgreSQL. Automates schema migrations, provides transactional ACID safety for payment ledgers, and eliminates raw SQL injection vulnerabilities.

SQLite / PostgreSQL [Relational Persistence Engines]

SQLite provides zero-config, portable embedded storage for offline edge mandi deployments. PostgreSQL handles enterprise production workloads requiring concurrent row-level locking.

Socket.IO (v4) [Bi-Directional Real-Time WebSocket Layer]

Drives live queue progression, instant token call alerts, and real-time silo capacity updates without requiring farmers or mandi managers to manually refresh their browser.

PDFKit [Server-Side Vector PDF Rendering Engine]

Generates official, tamper-evident Tulai Parchi (Weighment Slips) in pure JavaScript on the server. Produces lightweight vector documents with APMC watermarks, metadata, and financial breakdowns.

Telegraf & Telegram Bot API [Automated Messenger Integration]

Connects the mandi backend directly to farmers Telegram accounts. Dispatches instant notifications, queue alerts, and delivers PDF weighment slips directly to the farmer phone.

Zod [Runtime Schema Validation]

Validates inbound REST request payloads at the API perimeter. Guarantees that token bookings, assay results, and scheme uploads conform strictly to business invariants before touching the database.

JSON Web Tokens (JWT) & bcrypt [Stateless Authentication & Security]

Implements industry-standard stateless authentication. Passwords are encrypted with salt-based bcrypt hashing, and role-based permissions (Farmer, Manager, Admin) are enforced via JWT bearer tokens.

Lucide React [Consistent Vector Iconography]

Provides over 40 optimized SVG vector icons for intuitive agrarian interfaces, assisting farmers with varying literacy levels to navigate menus and visual cues effortlessly.

5. EXTERNAL APIS & INTEGRATION SERVICES

The platform bridges diverse external data services to give farmers and mandi managers comprehensive situational intelligence:

- **Telegram Bot API:** Used by `server/src/services/telegram.service.ts` to communicate with farmers in real time. It delivers interactive messages with inline reply buttons, dispatching the binary PDF Tulai Parchi directly to the chat as an official document.
 - **Dynamic Translation API (Google / MyMemory):** Used by `client/src/hooks/useDynamicTranslation.ts` and `server/src/controllers/translate.controller.ts`. Translates dynamic government circulars, advisories, and notifications on the fly into 10+ Indian languages (Hindi, Punjabi, Bengali, Marathi, Gujarati, Telugu, Tamil, etc.) with in-memory caching.
 - **OpenWeatherMap / Agronomy Weather API:** Used by `server/src/services/ai.service.ts` and displayed on FarmerWeatherCard. Provides temperature, humidity, wind velocity, and rainfall forecasts to assist farmers in timing crop harvest and grain drying before mandi delivery.
 - **Agmarknet & Government MSP Price Feeds:** Maintains daily floor prices across central and state mandis, calculating parity prices and ensuring farmers receive legitimate benchmark rates for wheat, paddy, mustard, pulses, and oilseeds.
 - **HTML5 Geolocation API:** Used on the client to automatically detect the farmer latitude/longitude and compute straight-line distance to nearby APMC procurement yards.
-

- **Web Speech Recognition & Synthesis API:** Integrated into the AI Assistant interface to allow non-literate farmers to ask questions via voice and listen to spoken agricultural advice in their native vernacular.

6. DATA INTEGRITY, CONCURRENCY & SECURITY STANDARDS

Because the platform governs physical grain logistics and direct financial benefit disbursements, strict engineering safeguards are built into every layer:

- **ACID Payment Transactions:** When an assay is finalized and weighment recorded, the token state change, payment record creation, and mandi capacity decrement are executed inside a single Prisma database transaction (\$transaction). If any step fails, the entire batch rolls back to prevent phantom records.
 - **Room-Based Socket Isolation:** WebSockets are organized into isolated mandi rooms (centre:CENTRE_ID). Gate actions at Ludhiana APMC do not trigger re-renders or data leakage on dashboards in Karnal or Indore, conserving network bandwidth.
 - **Role-Based Access Control (RBAC):** Strict middleware checks authenticate JWT tokens and authorize actions: only authenticated FARMERS can book tokens; only verified MANAGERS can operate weighing bays; and only ADMINS can publish or amend government welfare schemes.
 - **Sensitive Data Masking:** Bank account numbers and Aadhaar credentials are automatically masked on the client and in printed receipts (e.g. XXXXXX4021) to safeguard farmer privacy.
-

